Güvenilirlik Mühendisliği: Yöntemler ve Alternatifleri Haritası

ÖFY

October 2026

Özet

Kontrollü arıza üretme ve tersine mühendislik, tek başına bir yöntem değil, daha geniş bir mühendislik ailesinin iki üyesidir. Bu ailenin geri kalanı dört soruya cevap verir: arıza gerçekleşmeden önce nasıl öngörülür, güvenilirlik nasıl sayıya dökülür ve sınanır, üretimin tutarlılığı nasıl sağlanır, arıza gerçekleştikten sonra ondan nasıl öğrenilir. Her soruda yerleşik bir yöntem ve o yöntemin zayıf noktasını kapatan alternatifler bulunur: istatistiksel ömür tahmininin karşısında arıza fiziği, yedekliliğin karşısında ortak neden arızası analizi, tek kök neden arayışının karşısında sistem temelli kaza modelleri yer alır. Netflix örneğinde arıza üretmek işin yalnızca yarısıdır; diğer yarısı devre kesici, yedek cevap ve yük atma gibi kod desenleri ile kodu yazan ekibin onu işletmesine dayanan organizasyon yapısıdır. Yavaşlayan bir bağımlılık, tamamen çöken bir bağımlılıktan daha tehlikelidir ve sunucu kapatan bir araç bunu sınamaz. Ülkeler ve kurumlar arasındaki yetkinlik farkı ise yöntem bilgisinden çok, yazıya dökülmemiş üretim bilgisinin birikimiyle açıklanır; bu birikim dışarıdan alınamaz, yaparak oluşur.

Giriş

Bir sistemin dayanıklı olduğunu söyleyebilmek için üç ayrı şeyin bilinmesi gerekir: sistemin nasıl bozulabileceği, ne sıklıkla bozulacağı ve bozulduğunda ne olacağı. Tek bir yöntem bu üçünü birden cevaplamaz. Çalışan bir sistemde rastgele sunucu kapatmak üçüncü soruya cevap verir, ancak ilk ikisi hakkında bir şey söylemez. Aynı biçimde, bir ürünü söküp ölçmek onun biçimini verir, ancak neden o biçimde yapıldığını vermez.

Bu boşlukları dolduran yöntemler farklı disiplinlerde gelişmiştir. **Güvenilirlik mühendisliği** (*reliability engineering*), bir ürünün belirli bir süre boyunca arızasız çalışma olasılığıyla ilgilenir. **Kalite mühendisliği** (*quality engineering*), üretilen her birimin tasarıma uygun ve birbirine benzer çıkmasıyla ilgilenir. **Emniyet mühendisliği** (*safety engineering*), sistemin insana ve çevreye zarar vermemesiyle ilgilenir. **Dayanıklılık** (*resilience*) ise yazılım dünyasında, sistemin arıza altında hizmet vermeyi sürdürme yeteneğini anlatır. Bu dört kavram sık sık birbirinin yerine kullanılır, ancak aynı şey değildir: bir ürün kaliteli olup güvenilmez olabilir, güvenilir olup emniyetsiz olabilir.

Yaygın bir yanılgı, bu yöntemlerin birbirinin rakibi olduğu ve en iyisinin seçilmesi gerektiğidir. Gerçekte her yöntem belirli bir körlüğe sahiptir ve alternatifleri o körlüğü kapatmak için geliştirilmiştir.

Bu not yöntemlerin bir haritasını çıkarır; her birinin ne işe yaradığını, zayıf noktasını ve alternatifini verir. Yöntemlerin adım adım uygulanışı ve matematiksel ayrıntıları kapsam dışındadır.

Arızayı Önceden Düşünme

Deney yapmak pahalıdır ve yalnızca denenen arızayı gösterir. Bu nedenle deneyden önce, olası arızalar masa başında sistematik biçimde listelenir.

Parçadan sonuca: FMEA ve alternatifleri

FMEA (*Failure Mode and Effects Analysis*, Hata Türü ve Etkileri Analizi), sistemin her parçası için üç sorunun sorulduğu bir yöntemdir: bu parça nasıl bozulabilir, bozulursa ne olur, sonuç ne kadar ağırdır. Örneğin bir pompa için "sızdırma", "hiç çalışmama" ve "düşük basınç" ayrı hata türleri olarak yazılır ve her birinin sisteme etkisi değerlendirilir. Yöntem aşağıdan yukarıya çalışır: parçadan başlar, sisteme doğru gider.

FMEA'nın zayıf noktası, parçalara tek tek bakmasıdır. İki parçanın ikisi de doğru çalışırken aralarındaki etkileşimden doğan arızaları göremez. Alternatifleri şunlardır:

- **FMECA:** FMEA'ya kritiklik puanı eklenmiş hâlidir; hata türlerini önem sırasına dizer.
- **Ön tehlike analizi** (*Preliminary Hazard Analysis*, PHA): Tasarımın çok erken aşamasında, ayrıntı yokken yapılan kaba tehlike listesidir.
- **HAZOP** (*Hazard and Operability Study*): Süreç endüstrisinde kullanılır. "Daha fazla", "daha az", "ters yönde" gibi kılavuz kelimelerle her akışın tasarımdan nasıl sapabileceği sorulur.
- **STPA** (*System-Theoretic Process Analysis*): Parçalara değil, kontrol ilişkilerine bakar. Arızayı bir parçanın bozulması olarak değil, bir kontrol komutunun yanlış, eksik ya da geç verilmesi olarak ele alır (Leveson, 2011).
- **Ön-otopsi** (*pre-mortem*): Proje başarısız olmuş gibi düşünülür ve nedeni geriye doğru yazılır.

Sonuçtan nedene: hata ağacı ve alternatifleri

Hata ağacı analizi (*Fault Tree Analysis*, FTA) ters yönde çalışır. İstenmeyen bir sonuç tepeye yazılır, örneğin "sistem çöktü". Ardından bu sonuca yol açabilecek nedenler "ve" ile "veya" kapılarıyla dallandırılır. Yöntem 1960'ların başında Bell Laboratuvarları'nda füze fırlatma kontrol sistemleri için geliştirilmiştir.

Alternatifleri:

- **Olay ağacı analizi** (*Event Tree Analysis*, ETA): Bir başlangıç arızasından başlar ve koruma katmanlarının çalışıp çalışmamasına göre olası sonuçlara doğru dallanır.
- **Papyon diyagramı** (*bow-tie*): Solda hata ağacını, sağda olay ağacını tek resimde birleştirir.

- **Güvenilirlik blok diyagramı:** Parçaların seri mi paralel mi bağlı olduğunu göstererek sistemin toplam güvenilirliğini hesaplar.
- **Markov modelleri:** Sistemin durumlar arasında zamanla nasıl geçtiğini modeller; onarılabilen sistemlerde kullanılır.

Tek hata noktası ve ortak neden arızası

Tek hata noktası (*Single Point of Failure*, SPOF), tek başına bozulduğunda bütün sistemi durduran parçadır. Çözümü **yedeklilik**tir (*redundancy*): aynı işi yapan birden fazla parça bulundurulur.

Yedekliliğin kör noktası **ortak neden arızası**dır (*common cause failure*). İki yedek parça aynı güç kaynağına bağlıysa, aynı yazılım hatasını taşıyorsa ya da aynı odadaysa, tek bir olay ikisini birden devre dışı bırakır. Bunun çözümü **çeşitlilik**tir (*diversity*): aynı iş farklı tasarımla, farklı ekiple ya da farklı teknolojiyle iki kez yapılır.

Emniyet ilkeleri

- **Güvenli arıza** (*fail-safe*): Sistem bozulduğunda zararsız bir duruma geçer. Asansör arızalandığında durur.
- **Arızada çalışmaya devam** (*fail-operational*): Sistem bozulduğunda da görevini sürdürmek zorundadır. Uçuş hâlindeki bir uçak duramaz. Bu iki ilke tamamen farklı tasarımlar gerektirir.
- **Derinlemesine savunma** (*defense in depth*): Tek bir engele güvenilmez; arka arkaya birkaç bağımsız koruma katmanı konur.

Güvenilirliğin Ölçülmesi ve Sınanması

Ölçütler

MTBF (*Mean Time Between Failures*), iki arıza arasında geçen ortalama süredir. Yaygın biçimde kullanılır, ancak tek başına yanıltıcıdır; çünkü arızanın ne kadar sürdüğünü söylemez. Bu nedenle başka ölçütlerle birlikte okunur:

- **MTTR** (*Mean Time To Repair*): Ortalama onarım süresidir. Yazılım dünyasında genel eğilim, arızaları seyrekleştirmek yerine toparlanma süresini kısaltmaktır.
- **Kullanılabilirlik** (*availability*): Sistemin çalışır durumda olduğu sürenin toplam süreye oranıdır.
- **B10 ömrü:** Üretilen parçaların yüzde onunun bozulduğu süredir. Ortalama yerine erken arızaları öne çıkarır.

Küvet eğrisi ve ömür dağılımları

Küvet eğrisi (*bathtub curve*), arıza oranının zamanla nasıl değiştiğini gösterir. Eğrinin üç bölgesi vardır: ömrün başında üretim kusurlarından kaynaklanan yüksek arıza oranı, ortada düşük ve sabit bir oran, sonda yıpranmayla yeniden yükselen oran. **Weibull dağılımı** bu üç bölgeyi tek bir biçim parametresiyle ifade edebildiği için güvenilirlik analizinde standart araçtır.

İstatistiksel yaklaşımın zayıf noktası, arızanın nedenini açıklamamasıdır; yalnızca ne zaman olacağını tahmin eder. Alternatifi **arıza fiziği**dir (*physics of failure*). Bu yaklaşım, yorulma, korozyon ya da ısıl çevrim gibi bozulma mekanizmasını modelleyerek ömrü hesaplar. Yeni bir tasarımda geçmiş arıza verisi bulunmadığı için istatistik işe yaramaz, arıza fiziği yarar.

Hızlandırılmış testler

HALT (*Highly Accelerated Life Test*), ürünün sıcaklık ve titreşim altında tasarım sınırlarının çok ötesinde, adım adım artan gerilimle zorlanmasıdır. Amaç ürünün geçip geçmediğini görmek değil, nerede ve nasıl kırıldığını bulmaktır (Hobbs, 2000). Bu yönüyle donanımda kasıtlı arıza üretmenin doğrudan karşılığıdır. **HASS** (*Highly Accelerated Stress Screen*) aynı mantığın seri üretimdeki her birime uygulanan hafifletilmiş hâlidir.

HALT ömür tahmini vermez; yalnızca zayıf noktayı gösterir. Bu boşluğu kapatan yöntemler:

- **Nicel hızlandırılmış ömür testi** (ALT): Kontrollü gerilim altında arıza süreleri ölçülür ve normal koşullardaki ömür hesaplanır.
- **Çevresel gerilim eleme** (ESS) ve **yakma testi** (*burn-in*): Küvet eğrisinin baş kısmındaki erken arızaları müşteriye gitmeden önce ortaya çıkarır.

- **Güvenilirlik büyüme testi:** Test et, düzelt, yeniden test et döngüsüyle güvenilirliğin zaman içindeki artışını izler.

Tasarım payları

Derating, bir parçanın anma kapasitesinin altında çalıştırılmasıdır. 100 voltluk bir kondansatör 50 voltta kullanıldığında ömrü belirgin biçimde uzar. Alternatifi **gürbüz tasarım**dır (*robust design*): parçayı az zorlamak yerine, tasarım üretimdeki ve kullanımdaki değişkenliğe duyarsız hâle getirilir.

Bakım

Güvenilirlik merkezli bakım (*Reliability-Centered Maintenance*, RCM), her parçanın bakımını o parçanın arıza biçimine göre belirler. **Kestirimci bakım** parçayı takvime göre değil, ölçülen durumuna göre değiştirir; titreşim, sıcaklık ve yağ analizi gibi **durum izleme** verilerine dayanır.

Kalite Mühendisliği

Üretim tutarlılığı

İstatistiksel süreç kontrolü (*Statistical Process Control*, SPC), üretim hattından düzenli ölçüm alınması ve bu ölçümlerin kontrol grafiklerine işlenmesidir. Amaç, sürecin doğal dalgalanması ile gerçek bir sorunun işareti olan sapmayı ayırt etmektir. **Cpk** süreç yeterlilik indeksidir; sürecin tolerans aralığına ne kadar rahat sığdığını tek sayıyla verir. **Altı Sigma**, hata oranını milyonda birkaç düzeyine indirmeyi hedefleyen, ölçüme dayalı bir iyileştirme yaklaşımıdır. Japon sanayisinin kalite kültürü olarak anılan şeyin içeriği büyük ölçüde bu istatistiksel disiplindir (Deming, 1986).

Ölçüp düzeltmeye dayanan bu yaklaşımın alternatifleri, hatayı kaynağında önlemeyi hedefler:

- **Poka-yoke:** Hatanın yapılmasını fiziksel olarak imkânsız kılan tasarımdır. Yalnızca tek yönde takılabilen bir fiş buna örnektir (Shingo, 1986).
- **Jidoka:** Hata görüldüğünde hattın durdurulmasıdır. Hatalı parçanın sonraki istasyona geçmesi engellenir (Ohno, 1988).
- **Yalın üretim ve kaizen:** İsrafın sürekli ve küçük adımlarla azaltılmasıdır.
- **Kabul örneklemesi:** Partinin tamamı yerine bir örneğin muayene edilmesidir. Hatayı önlemez, yalnızca yakalar.

Ölçüme güven

Ölçüm sistemi analizi (*Measurement System Analysis*, MSA), ölçüm aletinin ve ölçen kişinin sonuca ne kadar değişkenlik kattığını belirler. Aynı parça farklı kişilerce ölçüldüğünde farklı sonuç çıkıyorsa, parçalar arasındaki fark ile ölçüm hatası ayırt edilemez. Tersine mühendislikte ilk tuzak budur: yanlış ölçülen bir parçadan çıkarılan her sonuç yanlıştır. MSA'yı **kalibrasyon zinciri** ve **ölçüm belirsizliği** hesabı tamamlar.

Doğrulama ve geçerleme

Doğrulama (*verification*) ürünün şartnameye uygun yapılıp yapılmadığını, **geçerleme** (*validation*) ise şartnamenin gerçek ihtiyacı karşılayıp karşılamadığını sorar. İlki "ürün doğru mu yapıldı", ikincisi "doğru ürün mü yapıldı" sorusudur. Doğrulamanın dört yolu vardır: muayene, analiz, gösterim ve test. Kalifikasyon bu çerçevenin bir parçasıdır. **Bağımsız doğrulama ve geçerleme** (IV&V), kontrolü ürünü geliştirmeyen ayrı bir ekibin yapmasıdır.

Sistem mühendisliğindeki **V modeli** bu ilişkiyi görselleştirir: sol kolda gereksinimler ayrıntılanır, sağ kolda her düzey kendi testiyle eşleşir. **Gereksinim izlenebilirliği**, her gereksinimin hangi testle kanıtlandığının kaydıdır. **Arayüz kontrolü** ayrıca önemlidir; arızaların büyük bölümü parçaların içinde değil, birleştikleri yerde ortaya çıkar.

Konfigürasyon ve izlenebilirlik

Konfigürasyon yönetimi, hangi ürünün hangi parça revizyonlarından oluştuğunun kaydını tutar. **İzlenebilirlik**, bir parçanın hangi üretim partisinden ve hangi malzemeden geldiğini geriye doğru takip etmeyi sağlar. Sahada bir arıza çıktığında aynı partiden başka hangi ürünlerin etkilendiği ancak bu kayıtla bulunur. Güncel karşılığı **ürün yaşam döngüsü yönetimi** (PLM) sistemleridir.

Ölçüm ve malzeme incelemesi

- **Metroloji:** Koordinat ölçüm makinesi ve üç boyutlu tarama ile parça geometrisinin çıkarılmasıdır. **GD&T**, geometrik toleransların çizimde ifade edildiği dildir.
- **Hasar analizi:** Kırılan parçanın incelenmesidir. Kırılma yüzeyi (*fraktografi*) ve iç yapı (*metalografi*), parçanın yorulmadan mı, aşırı yükten mi, malzeme kusurundan mı kırıldığını gösterir.
- **Tahribatsız muayene** (*Non-Destructive Testing*, NDT): Röntgen, ultrason ve tomografi ile parçanın kesilmeden iç kusurlarının görülmesidir.

Deney ve Model Kurma Yöntemleri

Sistem tanılama

Sistem tanılama (*system identification*), iç yapısı bilinmeyen bir sisteme bilinen bir giriş sinyali uygulanması ve çıkışından sistemin matematik modelinin çıkarılmasıdır (Ljung, 1999). Örneğin bir motora ani bir gerilim basamağı verilir ve hızının zamanla nasıl değiştiği kaydedilir; bu eğriden motorun ataleti ve sürtünmesi hesaplanır. Küçük bir sapma üretip gözlemden çıkarsama yapma döngüsünün mühendislikteki resmî adı budur. **Kararlılık payı** ise sistemin kontrolden çıkmasına ne kadar mesafe kaldığını ölçer.

Deney tasarımı

Deney tasarımı (*Design of Experiments*, DoE), hangi değişkenin hangi düzeyde ve hangi sırayla değiştirileceğinin önceden planlanmasıdır (Montgomery, 2017). Değişkenleri tek tek denemek hem çok deney gerektirir hem de değişkenler arasındaki etkileşimi kaçırır. Faktöriyel tasarım, birkaç değişkeni aynı anda değiştirerek az deneyle daha çok bilgi verir. Taguchi yöntemi bu yaklaşımı gürbüz tasarıma uygular.

Benzetim türleri

- **Döngüde donanım** (*Hardware-in-the-Loop*, HIL): Gerçek kontrol kartı, sahte bir çevre modeline bağlanarak test edilir. Uçuş bilgisayarı uçtuğunu sanır, ancak masadadır.
- **Dijital ikiz:** Sahadaki belirli bir sistemin, o sistemden gelen canlı veriyle güncellenen modelidir.

Yazılım testinde arıza üretme

- **Fuzzing:** Programa çok sayıda rastgele ya da bozuk girdi verilir ve nerede çöktüğü izlenir.
- **Mutasyon testi:** Koda kasıtlı küçük hatalar eklenir ve mevcut testlerin bunları yakalayıp yakalamadığına bakılır. Sınanan şey kod değil, testlerin kendisidir.
- **Jepsen testleri:** Dağıtık veritabanlarında ağ kasıtlı olarak bölünür ve veri kaybı ya da tutarsızlık olup olmadığı denetlenir. Arkasındaki kuram **CAP teoremi**dir: ağ bölündüğünde bir sistem tutarlılık ile erişilebilirlikten yalnızca birini koruyabilir (Gilbert ve Lynch, 2002).
- **Biçimsel doğrulama:** Tasarım çalıştırılmadan, matematiksel bir modelle bütün olası durumlar için kanıtlanır. Lamport'un geliştirdiği TLA+ dili bu amaçla kullanılır.

Güvenlik alanındaki karşılıklar

Kırmızı takım çalışması ve **sızma testi**, kasıtlı arıza üretmenin kötü niyetli bir saldırganı taklit eden biçimidir. Yazılımda tersine mühendislik ise derlenmiş bir programın geri açılarak okunmasıdır; zararlı yazılım analizinde ve güvenlik açığı araştırmasında kullanılır.

Yazılım Dayanıklılığı: Netflix Örneği

Çıkış noktası

Ağustos 2008'de Netflix'in kendi veri merkezindeki veritabanı bozulmuş ve şirket üç gün boyunca DVD gönderememiştir. O dönemde mimari, büyütülmüş tek makinelere dayanıyordu. Bu olay buluta geçiş kararının gerekçesi olmuş, kasıtlı arıza üreten araçlar da bu geçişin içinde doğmuştur (Izrailevsky ve Tseitlin, 2011).

Arıza üretiminin evrimi

Rastgele sunucu kapatan Chaos Monkey ilk adımdır. Netflix bunu **Simian Army** adıyla bir araç ailesine genişletmiştir:

Araç	Ürettiği arıza
Chaos Monkey	Tek bir sanal makineyi kapatır
Latency Monkey	Servisler arası çağrılara yapay gecikme ekler
Chaos Gorilla	Bütün bir erişilebilirlik bölgesini devre dışı bırakır
Chaos Kong	Bütün bir coğrafi bulut bölgesini devre dışı bırakır

Bu ailedeki en önemli ders, yavaşlığın çökmeden daha tehlikeli olduğudur. Kapanan bir sunucu hemen fark edilir ve trafik başka yere yönlenir. Yavaş cevap veren bir sunucu ise kendisini bekleyen bütün istekleri bekletir; bekleyen istekler kaynakları tüketir ve arıza zincirleme biçimde yayılır.

Sonraki araçlar daha hassas çalışır. **FIT** (*Failure Injection Testing*), sunucuyu kapatmak yerine belirli bir isteğe hata enjekte eder; böylece etki tek bir kullanıcı ya da cihazla sınırlanabilir. **ChAP** (*Chaos Automation Platform*), trafiğin küçük bir dilimini iki gruba ayırır, yalnızca birine hata verir ve iki grubu karşılaştırır. Bu, arıza deneyinin kontrol gruplu hâlidir. **Bölge tahliyesi** ise bir bulut bölgesindeki bütün trafiğin başka bölgeye taşınması ve bunun düzenli olarak prova edilmesidir.

Kaos mühendisliğinin ilkeleri

Netflix ekibi yöntemi beş ilkede toplamıştır (Basiri vd., 2016):

1. **Kararlı durum hipotezi kurulur.** Deneyden önce sistemin normal hâli ölçülebilir bir sayıyla tanımlanır. Netflix'te bu sayı saniyede başlatılan oynatma sayısıdır.

2. **Gerçek dünyadaki olaylar taklit edilir.** Sunucu kaybı, ağ gecikmesi, disk dolması gibi gerçekten yaşanan arızalar üretilir.

3. **Deney üretim ortamında yapılır.** Test ortamı gerçek trafiği ve gerçek bağımlılıkları yansıtmaz.

4. **Deneyler otomatikleştirilir.** Elle yapılan deney seyrek kalır ve sistem değiştikçe geçerliliğini yitirir.

5. **Etki alanı küçük tutulur. Etki alanı** (*blast radius*), deneyin zarar verebileceği kullanıcı kitlesidir. Deney kararlı durumu bozuyorsa derhâl durdurulur.

Üretimde otomatik deneyin alternatifleri de vardır: **oyun günü** (elle ve planlı yapılan arıza provası), **gölge trafik** (gerçek isteklerin kopyasının yeni sürüme gönderilmesi) ve **özellik bayrağı** (yeni davranışın koddan değil ayardan açılıp kapatılması).

Arızaya dayanmayı sağlayan desenler

Arıza üreten araç zayıflığı bulur, ancak gidermez. Giderme, kodda uygulanan desenlerle yapılır (Nygard, 2007):

- **Devre kesici** (*circuit breaker*): Bir bağımlılık art arda hata verdiğinde ona istek göndermek geçici olarak kesilir. Netflix bu deseni Hystrix kütüphanesiyle yaygınlaştırmıştır.
- **Yedek cevap** (*fallback*): Bağımlılık cevap vermediğinde daha basit bir cevap döndürülür. Kişisel öneri servisi çöktüğünde genel popüler liste gösterilir.
- **Zaman aşımı ve yeniden deneme:** Her çağrıya süre sınırı konur. Yeniden denemeler yanlış ayarlandığında zaten zorlanan servise yük bindirir ve sistemi kendi trafiğiyle boğar; bu nedenle denemeler arasındaki süre giderek uzatılır.
- **Bölmeleme** (*bulkhead*): Her bağımlılığa ayrı kaynak havuzu verilir; biri tıkandığında diğerleri etkilenmez.
- **Yük atma** (*load shedding*): Aşırı yükte önemsiz istekler reddedilir, kritik işlev korunur.
- **Durumsuz servis:** Sunucu üzerinde kalıcı veri tutulmaz. Rastgele sunucu kapatmanın mümkün olmasının ön şartı budur.

Devre kesicinin alternatifleri arasında **geri basınç** (yavaş tüketicinin üreticiyi yavaşlatması), **hız sınırlama** ve **yedekli istek** (aynı isteğin iki sunucuya gönderilip ilk cevabın alınması; Dean ve Barroso, 2013) bulunur.

Yayınlama ve gözlem

Kanarya analizi, yeni sürümün önce trafiğin küçük bir dilimine verilmesi ve eski sürümle ölçümler üzerinden otomatik karşılaştırılmasıdır. **Gözlemlenebilirlik** (*observability*), sistemin iç durumunun dışarıdan ölçümlerle anlaşılabilmesidir; ölçülemeyen bir sistemde deney yapılamaz, çünkü sonucu görülemez.

SLO (*Service Level Objective*), bir servisin hedeflenen güvenilirlik düzeyidir. **Hata bütçesi**, bu hedefin izin verdiği arıza miktarıdır: hedef yüzde 99,9 ise kalan binde bir harcanabilir bütçedir. Bütçe bittiğinde yeni özellik yayını durur, güvenilirlik çalışması öne alınır (Beyer vd., 2016).

Organizasyon

Araçların etkisi organizasyon yapısına bağlıdır. Netflix'te kodu yazan ekip aynı zamanda onu üretimde işletir ve nöbetini tutar. Bu yapıda rastgele kapatılan sunucu, kodu yazan kişinin kendi sorunudur; dayanıklı kod yazmak için doğrudan bir neden oluşur. Dayanıklılık desenleri hazır kütüphaneler ve ortak altyapı olarak sunulur; ekipler bunları sıfırdan yazmaz. Arıza sonrasında **suçlamasız olay incelemesi** (*blameless postmortem*) yapılır; aranan suçlu değil, arızayı mümkün kılan koşuldur.

Araçların bugünkü durumu

Simian Army'nin araçlarının çoğu emekliye ayrılmış, Chaos Monkey yayın hattına bağlı ayrı bir araç olarak sürdürülmüş, Hystrix ise bakım moduna alınmıştır. Değişmeyen kısım araçlar değil ilkelerdir.

Arızadan Öğrenme

Kök neden yöntemleri

5 Neden, bir arızanın ardından "neden" sorusunun art arda sorulmasıdır. **Balık kılçığı diyagramı** olası nedenleri insan, makine, malzeme ve yöntem gibi başlıklar altında toplar. Her ikisi de basittir, ancak tek bir kök neden bulunduğunu varsayar; karmaşık sistemlerde bu varsayım çoğu zaman yanlıştır.

Daha yapılandırılmış alternatifler: **8D raporu** (sekiz adımlı sorun çözme disiplini), **A3 düşünme** (sorunun tek sayfada özetlenmesi), **Kepner-Tregoe** yöntemi ve **FRACAS** (*Failure Reporting, Analysis and Corrective Action System*; her arızanın kaydedildiği ve düzeltici faaliyetin kapanışa kadar izlendiği sistem).

Kaza modelleri

- **İsviçre peyniri modeli:** Her koruma katmanı delikli bir peynir dilimi gibidir. Kaza, deliklerin aynı hizaya geldiği anda gerçekleşir (Reason, 1990).
- **Normal kazalar:** Parçaları sıkı bağlı ve etkileşimleri karmaşık olan sistemlerde kaza kaçınılmazdır; bu sistemlere koruma eklemek karmaşıklığı da artırır (Perrow, 1984).
- **STAMP:** Kaza bir olaylar zinciri değil, bir kontrol eksikliğidir. Hiçbir parça bozulmadan da kaza olabilir (Leveson, 2011).
- **Sapmanın normalleşmesi:** Küçük kural dışılıklar sonuçsuz kaldıkça kabul görür ve sınır fark edilmeden kayar (Vaughan, 1996).
- **Emniyet-II:** Yalnızca neyin yanlış gittiğine değil, işlerin neden çoğu zaman doğru gittiğine bakar (Hollnagel, 2014).

İnsan ve organizasyon

İnsan faktörleri mühendisliği, operatörün hata yapmasını kolaylaştıran tasarım kusurlarını inceler; çok sık çalan alarmların dikkate alınmaması (*alarm yorgunluğu*) tipik bir örnektir. Kontrol listeleri ve havacılıktan gelen ekip kaynak yönetimi bu alanın araçlarıdır. **Yüksek güvenilirlikli organizasyonlar** (*High Reliability Organizations*, HRO), uçak gemisi ve nükleer santral gibi hata kaldırmayan kurumların ortak alışkanlıklarını tanımlar (Weick ve Sutcliffe, 2001). Havacılıktaki **olay raporlama sistemleri** ise ucuz atlatılan olayların cezasız bildirilmesini sağlar; böylece kaza olmadan veri toplanır.

Bilgi Edinimi ve Kavramsal Çerçeve

Örtük bilgi

Örtük bilgi (*tacit knowledge*), sahibinin tam olarak söze dökemediği bilgidir (Polanyi, 1966). Bir kaynakçının iyi dikişi nasıl attığı ya da bir dökümcünün kalıbı ne zaman açacağı belgelerde yazmaz. Sökülen bir üründen okunamayan bilgi budur. Nonaka ve Takeuchi (1995), örtük bilginin kurum içinde açık bilgiye nasıl çevrildiğini ve yeniden örtük hâle nasıl geldiğini modellemiştir. Collins (2010) ise örtük bilginin bazı türlerinin ilke olarak yazıya dökülemeyeceğini savunur.

Bilginin birikmesi

Özümseme kapasitesi (*absorptive capacity*), bir kurumun dışarıdaki bilgiyi tanıma, alma ve kullanma yeteneğidir; bu yetenek kurumun önceden biriktirdiği bilgiye bağlıdır (Cohen ve Levinthal, 1990). Yani dışarıdan bilgi alabilmek için içeride zaten bilgi bulunması gerekir. **Yaparak öğrenme** (*learning by doing*) ise üretim bilgisinin üretim miktarıyla birlikte arttığını söyler (Arrow, 1962). Bu iki kavram birlikte, ileri teknolojide geriden gelen bir ekosistemin neden yalnızca inceleyerek yetişemediğini açıklar.

Tedarik zinciri bu tabloya siyasi bir boyut ekler. **Çift kaynak**, kritik bir parçanın iki ayrı tedarikçiden alınmasıdır. **İhracat kontrolü** ise parçanın üretilmeye devam ettiği hâlde satılmadığı durumu yaratır.

Dayanıklılığın ötesi

Kırılganlık karşıtlığı (*antifragility*), bir sistemin darbeden yalnızca sağ çıkmasını değil, darbe sayesinde güçlenmesini anlatır (Taleb, 2012). Düzenli kasıtlı arıza üreten bir kurum bu fikri uygular. **Dayanıklılık mühendisliği** (*resilience engineering*) ise emniyeti arızaların yokluğu olarak değil, sistemin beklenmedik duruma uyum sağlama yeteneği olarak tanımlar (Hollnagel, Woods ve Leveson, 2006).

Standartlar

Kalifikasyonun neye göre yapılacağını standartlar belirler:

Standart	Konusu
MIL-STD-810	Çevre koşulları testleri
MIL-STD-461	Elektromanyetik uyumluluk
MIL-STD-882	Sistem emniyeti
AS9100	Havacılık ve savunmada kalite yönetimi
AQAP	NATO kalite güvence gereksinimleri
DO-178C	Uçuş yazılımı
DO-254	Uçuş elektronik donanımı
DO-160	Uçak ekipmanı çevre testleri
IEC 61508, ISO 26262	İşlevsel emniyet (genel ve otomotiv)

Başka alanlardaki karşılıklar

Aynı mantık mühendislik dışında da görülür. Bankacılıkta **stres testi**, kurumun varsayımsal bir kriz senaryosu altında ayakta kalıp kalmadığını hesaplar. Tıpta aşı, küçük ve kontrollü bir uyarıyla bağışıklık kurar. Biyolojide gen susturma deneyleri, bir genin kapatılıp organizmada neyin bozulduğuna bakılmasıyla o genin işlevini ortaya çıkarır.

Sonuç

Yöntemlerin tamamı aynı iki hamlenin çeşitlemeleridir: arızayı önceden hayal etmek ve arızayı kasıtlı olarak üretmek. FMEA ile hata ağacı birincinin, HALT ile kaos deneyi ikincinin araçlarıdır. Bundan çıkan pratik sonuç, hiçbirinin tek başına yeterli olmadığıdır. Yalnızca analiz yapan kurum hayal edemediği arızaya yakalanır; yalnızca deney yapan kurum denemediği arızaya.

Alternatiflerin ortak yönü, yerleşik yöntemin gizli varsayımını hedef almalarıdır. İstatistik geçmiş veriyi, yedeklilik parçaların bağımsız bozulduğunu, 5 Neden tek bir kök nedeni varsayar. Bir yöntem seçilirken sorulması gereken soru, o yöntemin hangi varsayıma dayandığı ve o varsayımın eldeki sistemde geçerli olup olmadığıdır.

Netflix örneği, aracın tek başına bir şey değiştirmediğini gösterir. Rastgele sunucu kapatmak, arızanın bedelini kodu yazan ekibe yükleyen bir organizasyon ve arızayı karşılayacak hazır desenler olmadan yalnızca kesinti üretir. Aynı durum donanım tarafında da geçerlidir: test laboratuvarı, arıza kayıt sistemi ve düzeltici faaliyet döngüsü olmayan bir kurumda HALT yalnızca kırık parça üretir.

Çerçevenin sınırı, bir harita olmasıdır. Yöntemlerin hangi sırayla ve hangi ölçekte uygulanacağı sektöre, ürünün ömrüne ve arızanın bedeline göre değişir; küçük bir ekibin bu listenin tamamını uygulaması ne mümkündür ne gereklidir. Netflix'e ait araç ve tarih bilgileri şirketin kendi yayınlarına dayanır ve güncel durumları değişmiş olabilir.

Açık kalan soru önceliktir: sınırlı kaynağı olan bir kurumun bu yöntemlerden hangisine önce yatırım yapması gerektiği ve bu seçimin, örtük bilginin birikme hızını nasıl etkilediği bu notta cevaplanmamıştır.

Kaynaklar

- Arrow, K. J. (1962). The Economic Implications of Learning by Doing. *The Review of Economic Studies*, 29(3).
- Basiri, A., Behnam, N., de Rooij, R., Hochstein, L., Kosewski, L., Reynolds, J., Rosenthal, C. (2016). Chaos Engineering. *IEEE Software*, 33(3).
- Beyer, B., Jones, C., Petoff, J., Murphy, N. R. (ed.) (2016). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly.
- Cohen, W. M., Levinthal, D. A. (1990). Absorptive Capacity: A New Perspective on Learning and Innovation. *Administrative Science Quarterly*, 35(1).
- Collins, H. (2010). *Tacit and Explicit Knowledge*. University of Chicago Press.
- Dean, J., Barroso, L. A. (2013). The Tail at Scale. *Communications of the ACM*, 56(2).

- Deming, W. E. (1986). *Out of the Crisis*. MIT Press.
- Gilbert, S., Lynch, N. (2002). Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services. *ACM SIGACT News*, 33(2).
- Hobbs, G. K. (2000). *Accelerated Reliability Engineering: HALT and HASS*. Wiley.
- Hollnagel, E. (2014). *Safety-I and Safety-II: The Past and Future of Safety Management*. Ashgate.
- Hollnagel, E., Woods, D. D., Leveson, N. (ed.) (2006). *Resilience Engineering: Concepts and Precepts*. Ashgate.
- Izrailevsky, Y., Tseitlin, A. (2011). The Netflix Simian Army. *Netflix Technology Blog*.
- Leveson, N. G. (2011). *Engineering a Safer World: Systems Thinking Applied to Safety*. MIT Press.
- Ljung, L. (1999). *System Identification: Theory for the User* (2. baskı). Prentice Hall.
- Montgomery, D. C. (2017). *Design and Analysis of Experiments* (9. baskı). Wiley.
- Nonaka, I., Takeuchi, H. (1995). *The Knowledge-Creating Company*. Oxford University Press.
- Nygard, M. T. (2007). *Release It! Design and Deploy Production-Ready Software*. Pragmatic Bookshelf.
- Ohno, T. (1988). *Toyota Production System: Beyond Large-Scale Production*. Productivity Press.
- Perrow, C. (1984). *Normal Accidents: Living with High-Risk Technologies*. Basic Books.
- Polanyi, M. (1966). *The Tacit Dimension*. Doubleday.
- Reason, J. (1990). *Human Error*. Cambridge University Press.
- Shingo, S. (1986). *Zero Quality Control: Source Inspection and the Poka-Yoke System*. Productivity Press.
- Taleb, N. N. (2012). *Antifragile: Things That Gain from Disorder*. Random House.
- Vaughan, D. (1996). *The Challenger Launch Decision*. University of Chicago Press.

- Weick, K. E., Sutcliffe, K. M. (2001). *Managing the Unexpected*. Jossey-Bass.

FIT, ChAP, kanarya analizi ve ekip yapısına ilişkin bilgiler Netflix Technology Blog yazılarına dayanmaktadır; bu yazıların tam başlıkları ve tarihleri doğrulanmalıdır.